# 6. Neural Networks, Part 2

Seeing and remembering: CNNs and sequence models

Week 6 · CE/ARCE 2XX AI Literacy for Civil and Architectural Engineering



COLLEGE OF ENGINEERING
Civil & Architectural Engineering & Mechanics

Agenda

# Today's plan

1. Quiz on last week (Week 5)
2. Architecture = built-in assumptions
3. Seeing: convolution and CNNs
4. Transfer learning and vision in CE/ARCE
5. Remembering: sequences and time series
6. Attention, and choosing an architecture
7. Lab 06, HW3, and the midterm

Recap

# Last week in 60 seconds

## 1 Neuron

Weighted sum plus an activation: $z = w_1x_1 + w_2x_2 + \dots + b$, then $a = g(z)$. ReLU is the default bend.

## 2 Forward pass

Multiply, add, bend, layer by layer. Our 2–2–1 example gave $\hat{y} = -1.5$ with 9 parameters.

## 3 Training

Loss + gradient descent. Backpropagation passes the blame backward to every weight.

## 4 Overfitting

Training error falls while test error rises: stop early, simplify, and compare with a baseline.

Learning objectives

# By the end of today, you can…

- ✓ Explain why a network's architecture is a set of built-in assumptions about the data
- ✓ Describe how convolution, pooling, and stacked layers let a CNN find cracks in photos
- ✓ Explain transfer learning and why it suits small engineering datasets
- ✓ Turn a time series into training examples with a sliding window; say what an RNN or LSTM adds
- ✓ Choose a suitable architecture — MLP, CNN, RNN, or transformer — for a CE/ARCE task

01

# Architecture = assumptions

Different data, different wiring

Architecture = assumptions

# A network's architecture is a set of built-in assumptions

Like a structural system: a truss assumes axial forces, a moment frame assumes bending

**TABLES OF NUMBERS**

## MLP

Any input may matter to any neuron; the order of the columns means nothing.

**IMAGES AND GRIDS**

## CNN

Nearby pixels belong together; a crack looks like a crack anywhere in the photo.

**TIME SERIES**

## RNN / LSTM

Order matters; the recent past helps predict the next step.

**TEXT, LONG SEQUENCES**

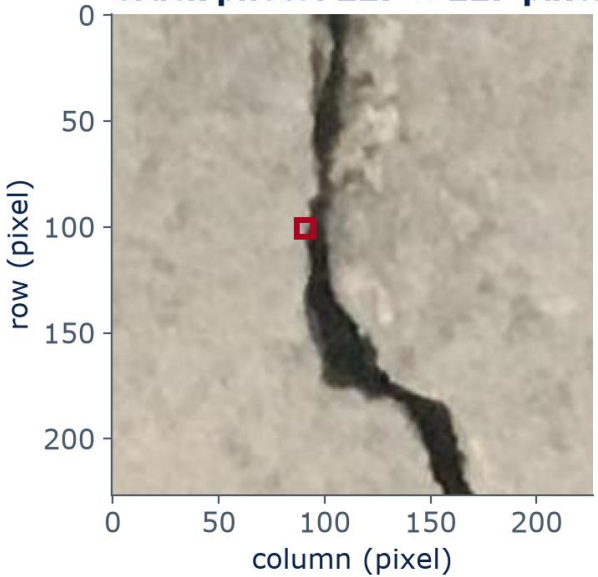
## Transformer

Anything may relate to anything; learn which parts to pay attention to.

Architecture = assumptions

# To a computer, a photo is just a grid of numbers

**Crack photo: 227 × 227 pixels**




**Zoom: the red 8 × 8 square, brightness 0–255**

| | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 |
|---|---|---|---|---|---|---|---|---|
| 97 | 185 | 176 | 167 | 162 | 166 | 153 | 103 | 59 |
| 98 | 184 | 176 | 167 | 157 | 159 | 149 | 104 | 62 |
| 99 | 182 | 175 | 167 | 156 | 152 | 140 | 99 | 58 |
| 100 | 179 | 172 | 170 | 158 | 145 | 126 | 87 | 50 |
| 101 | 178 | 172 | 175 | 162 | 136 | 108 | 71 | 42 |
| 102 | 179 | 174 | 181 | 165 | 128 | 90 | 56 | 40 |
| 103 | 180 | 175 | 185 | 167 | 122 | 78 | 47 | 40 |
| 104 | 183 | 180 | 182 | 162 | 117 | 71 | 50 | 47 |

column (pixel)

A color photo = 3 grids (red, green, blue): 227 × 227 × 3 = **154,587 numbers**

Architecture = assumptions

# Feeding raw pixels to an MLP explodes the number of weights

**154,587**

numbers in one 227 × 227 color photo

**≈ 155 million**

weights for just ONE hidden layer of 1,000 neurons on those inputs

**27 + 1**

weights + bias in one 3 × 3 color filter, reused at every position

154,587 × 1,000 = 154.6 million. Fewer weights → less data needed and less overfitting.

02

# Seeing: convolution and CNNs

Small filters that slide across the image

# Convolution: slide a small filter over the image, multiply, add

**Image patch (brightness 0–9)**

| | | | | | |
|---|---|---|---|---|---|
| 9 | 9 | 9 | 1 | 1 | 9 |
| 9 | 9 | 9 | 1 | 1 | 9 |
| 9 | 9 | 9 | 1 | 1 | 9 |
| 9 | 9 | 9 | 1 | 1 | 9 |
| 9 | 9 | 9 | 1 | 1 | 9 |

×

**Filter (weights)**

| | | |
|---|---|---|
| −1 | 0 | 1 |
| −2 | 0 | 2 |
| −1 | 0 | 1 |

=

**Feature map**

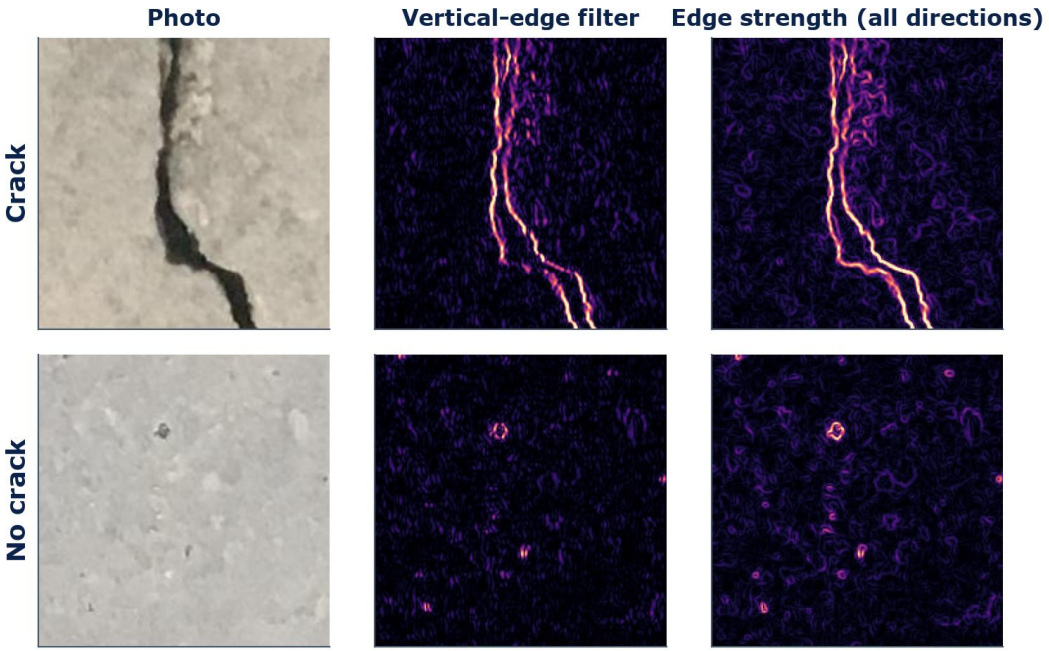
| | | | |
|---|---|---|---|
| 0 | −32 | −32 | 32 |
| 0 | −32 | −32 | 32 |
| 0 | −32 | −32 | 32 |

**Red window: (−9 + 1) + 2·(−9 + 1) + (−9 + 1) = −32**

big − : bright → dark edge
big + : dark → bright edge
0 : flat, no edge

The sign tells the direction of the change; the size tells the edge strength. In a CNN the filter's numbers are **weights learned by training**.

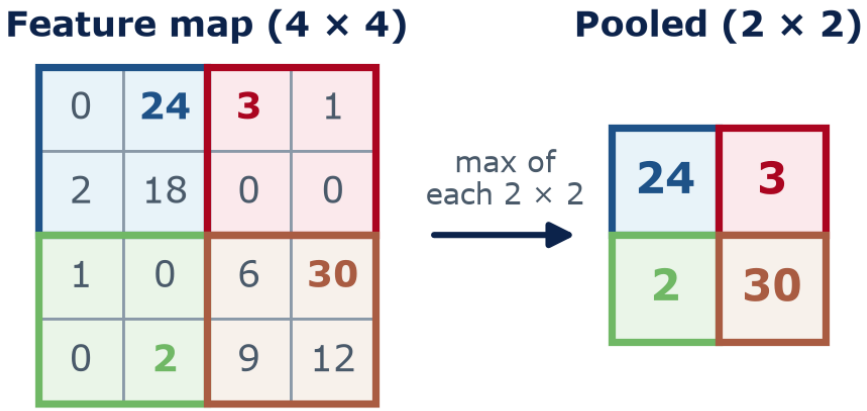
# On a real crack photo, an edge filter lights up the crack



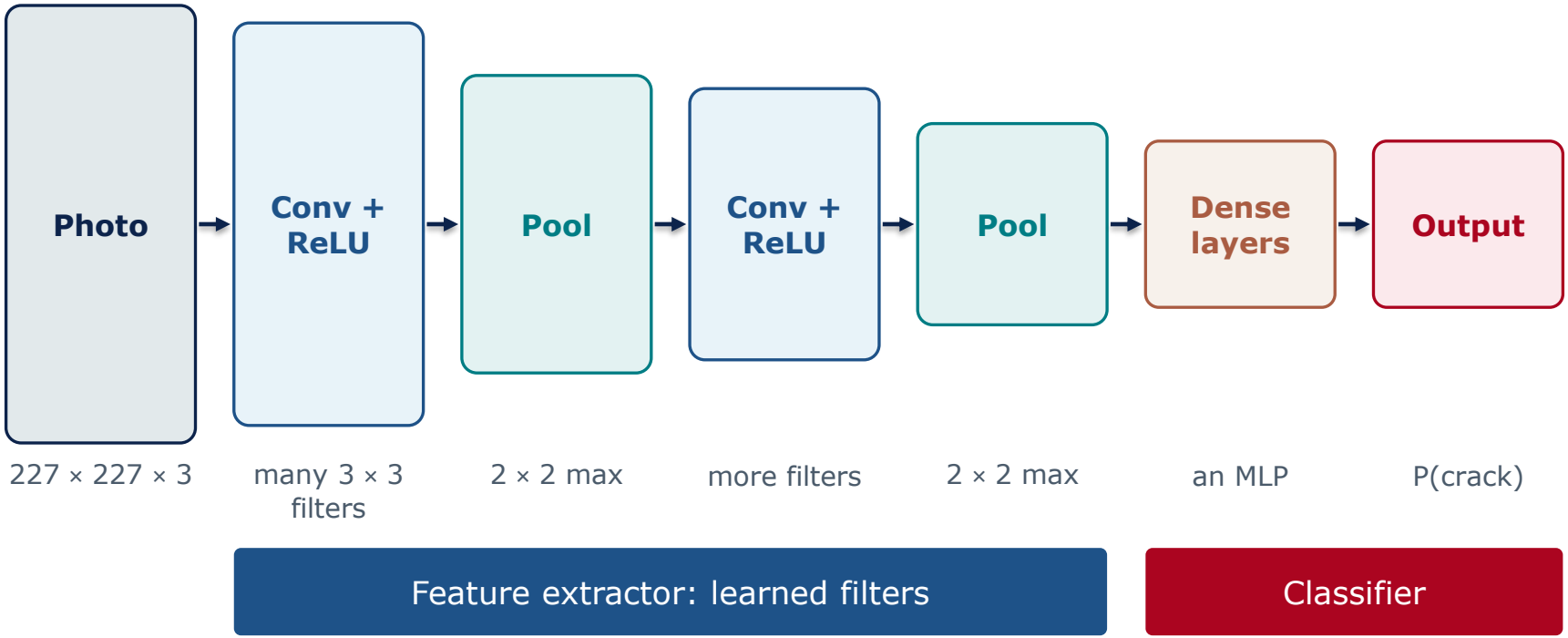

- A 3 × 3 edge filter, applied to real photos (computed with SciPy)
- **Crack:** both crack edges light up brightly
- **No crack:** only pores and spots respond, weakly
- The output is a **feature map**: where this pattern appears
- A CNN **learns** dozens to hundreds of filters per layer

# Pooling keeps the strongest signal and shrinks the map

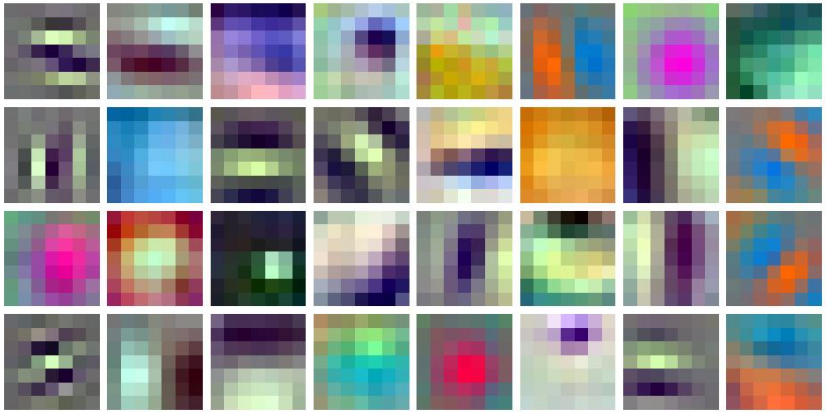
- **Max pooling:** keep the largest value in each 2 × 2 block
- Keeps the evidence that "there is an edge around here"
- Tolerates the crack shifting by a pixel or two
- 4× fewer numbers for the next layer to process



# A CNN stacks convolution and pooling, then a small classifier



# Layers learn a ladder: edges → textures → parts



32 of the 64 real first-layer filters of ResNet-18, learned from ImageNet photos

**Early layers**

Edges and color contrasts at every angle

**Middle layers**

Textures and patterns: pores, aggregate, rebar mesh, crack segments

**Late layers**

Parts and objects: a spall, a window, a hard hat

Concept check

# Why does a CNN need so few weights?

**Compared with an MLP on the same photo, why does a CNN need far fewer weights?**

**A** It ignores the color channels

**B** It reuses the same small filters at every position in the photo

**C** It only looks at the center of the photo

**D** It uses ReLU instead of sigmoid

Concept check

# Why does a CNN need so few weights? — answer

## Compared with an MLP on the same photo, why does a CNN need far fewer weights?

**A** It ignores the color channels

**B** It reuses the same small filters at every position in the photo

**C** It only looks at the center of the photo

**D** It uses ReLU instead of sigmoid

Weight sharing: one 3 × 3 filter slides over the whole image instead of one weight per pixel per neuron.

# 03

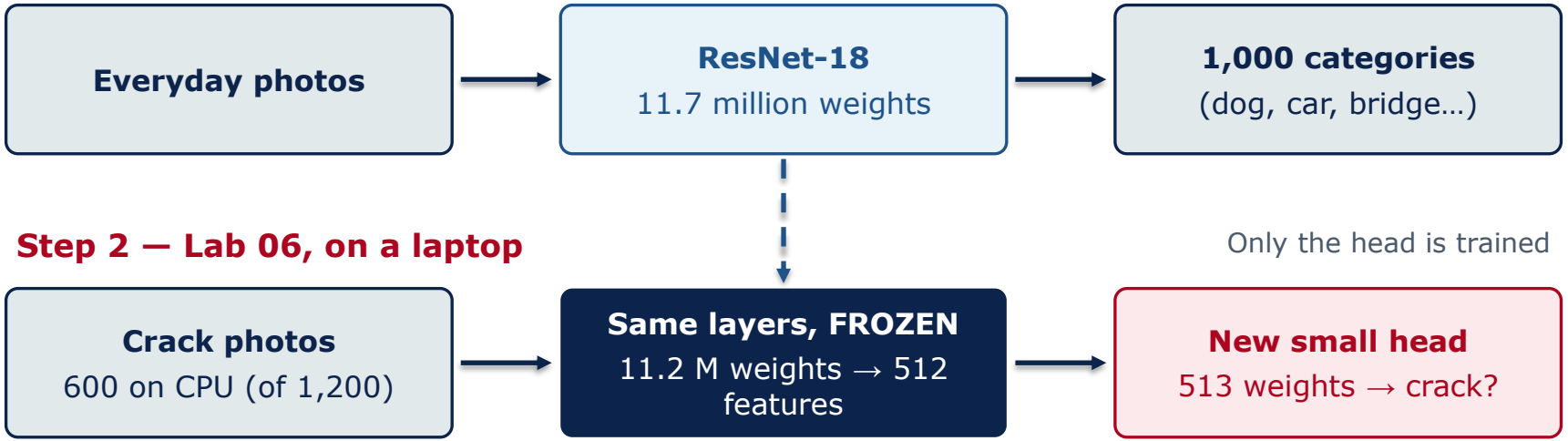
# Transfer learning and CE vision

Reuse what a network already learned

# Transfer learning: reuse a network pretrained on millions of photos

**Step 1 — done once by researchers**

ImageNet: ≈ 1.28 million labeled photos




Like hiring an experienced inspector and teaching only the specifics of your structure

# The Lab 06 recipe: pretrained features + a small head

```
import torch
from torchvision.models import resnet18, ResNet18_Weights
from sklearn.linear_model import LogisticRegression

weights = ResNet18_Weights.DEFAULT   # ImageNet-trained
model = resnet18(weights=weights).eval()
model.fc = torch.nn.Identity()       # drop last layer
prep = weights.transforms()          # resize, normalize

with torch.no_grad():                # no training here
    x = torch.stack([prep(im) for im in photos])
    F = model(x).numpy()             # 512 numbers/photo
head = LogisticRegression(max_iter=1000).fit(F, labels)
```

- Pretrained layers stay **frozen**: never trained
- Each photo → 512 numbers describing edges and textures
- Only the head learns: 512 weights + 1 bias
- Lab 06: accuracy, confusion matrix, and the mistakes

# Computer vision is already at work on bridges, sites, and satellites

| Application | Input | Output | Task type |
|---|---|---|---|
| **Crack and spall detection** | Drone or phone photos of bridges, tunnels, walls | Crack yes/no, or its outline | Classification, segmentation |
| **Post-disaster damage** | Aerial or satellite images before and after an event | Damage level for each building | Classification |
| **Construction-site safety** | Site camera frames | Boxes around workers missing hard hats or vests | Object detection |
| **Land use from satellites** | Satellite image tiles | Roof, road, vegetation, water per pixel | Segmentation |
| **Pavement condition** | Road images from survey vans | Crack type and severity | Detection, classification |

Transfer learning

# Would you deploy it on Arizona bridges?

**4 min**

**Think · Pair · Share**

- A crack classifier was trained on 1,200 close-up photos of walls on **one university campus**
- It scores very well on its own test set
- An agency wants to run it on **drone photos of Arizona bridge decks**
- List **three** things that could make it fail — and **one** test you would run first

Think about: sun and shadows, stains, joints and saw cuts, camera distance, dust, a different concrete texture, and the cost of a missed crack.

# Reading a confusion matrix: accuracy, precision, and recall

| | Model says → Crack | Model says → No crack |
|---|---|---|
| **Actually crack** | **92** true positive (TP) | **8** false negative (FN): missed crack |
| **Actually no crack** | **4** false positive (FP): false alarm | **96** true negative (TN) |

Example test set: 100 crack + 100 no-crack photos

**Accuracy = (TP + TN) / all**

share of all photos labeled correctly: 188/200 = 94%

**Precision = TP / (TP + FP)**

of the photos flagged "crack", how many really are? 92/96 ≈ 96%

**Recall = TP / (TP + FN)**

of the real cracks, how many did we catch? 92/100 = 92%

**Screening → recall matters most.** A missed crack can hide damage; a false alarm costs a second look. A lower threshold (flag if P(crack) > 0.3, not 0.5) raises recall but adds false alarms.

**1-minute check:** a bridge deck has 50 cracked photos. The model flags 40 photos; 30 of them are real cracks. Precision? Recall? Which worries you?

# 04

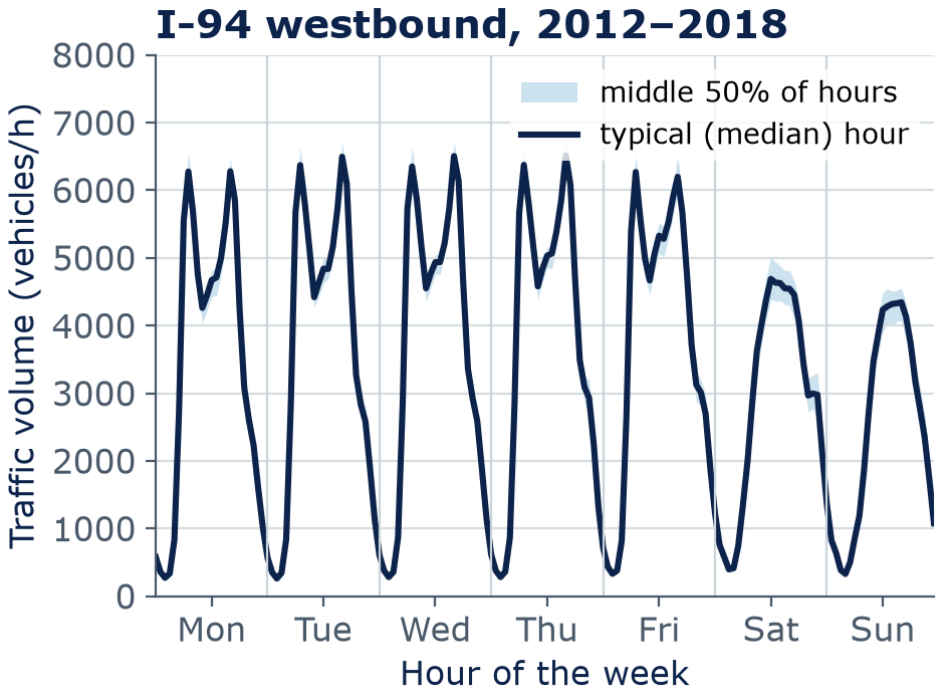
# Remembering: sequences

When the order of the data matters

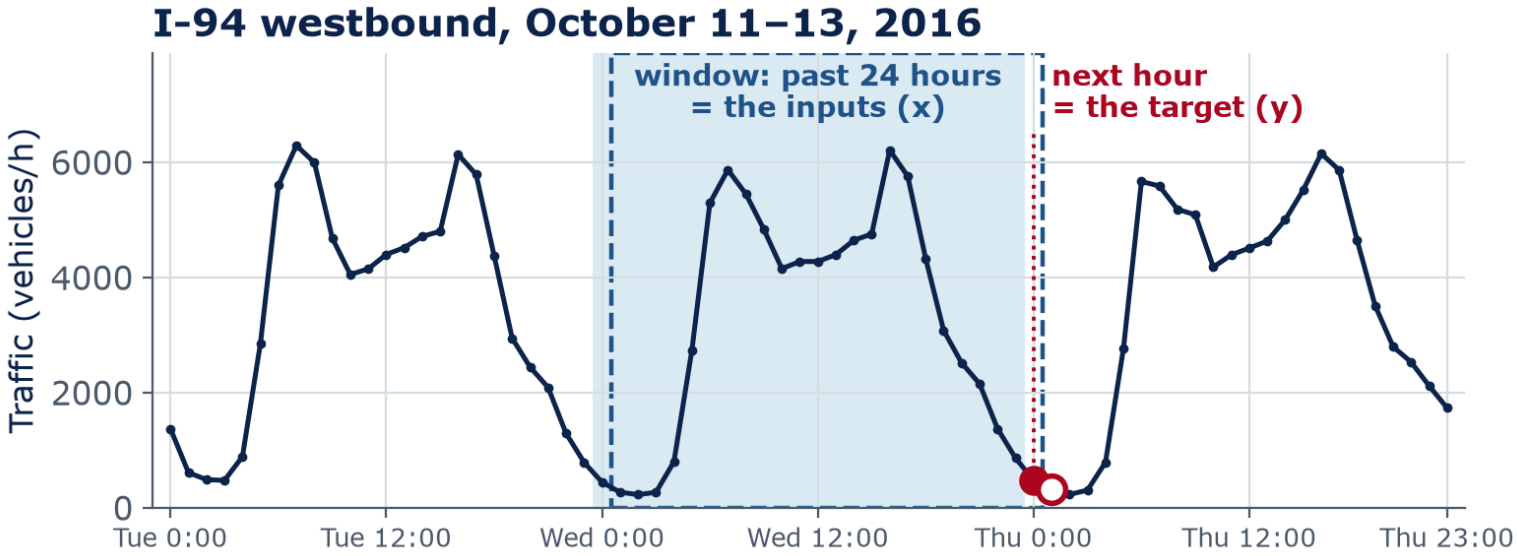
Remembering: sequences

# Many engineering signals are sequences with strong patterns

- In a **sequence**, the order of the values carries information
- Weekdays: morning and evening rush hours. Weekends: one midday hump
- The last few hours, and the same hour last week, help predict the next hour
- Same idea for building loads, river flow, and bridge vibration sensors



# A sliding window turns a time series into training examples



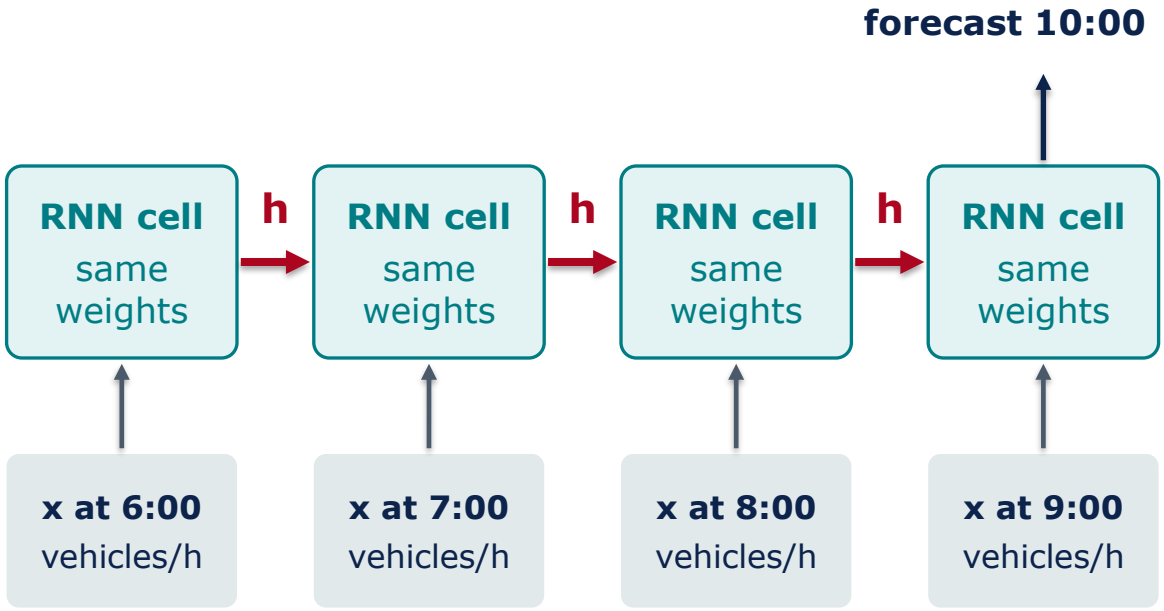

**Inputs x:** the 24 hourly volumes before time t

**Target y:** the volume at hour t (vehicles/h)

**Slide 1 hour:** → a new example; repeat thousands of times

Remembering: sequences

# Recurrent networks read one step at a time and carry a memory




Red arrows: the hidden state h — the network's memory

- Reads in order, one step at a time
- Carries a **hidden state h**: a running summary
- Reuses the same weights at every step
- **LSTM:** gates decide what to keep and forget → longer memory

Remembering: sequences

# Sequence models forecast traffic, building loads, and river flow

TRANSPORTATION

## Traffic volume

- Next-hour or next-day volumes on a corridor
- Inputs: past volumes, time, weather, events
- Uses: signal timing, travel-time information

BUILDING SYSTEMS (ARCE)

## Building load

- Tomorrow's hourly heating, cooling, or electric load
- Inputs: past load, weather forecast, schedule
- Uses: HVAC scheduling, demand response

WATER RESOURCES

## River flow

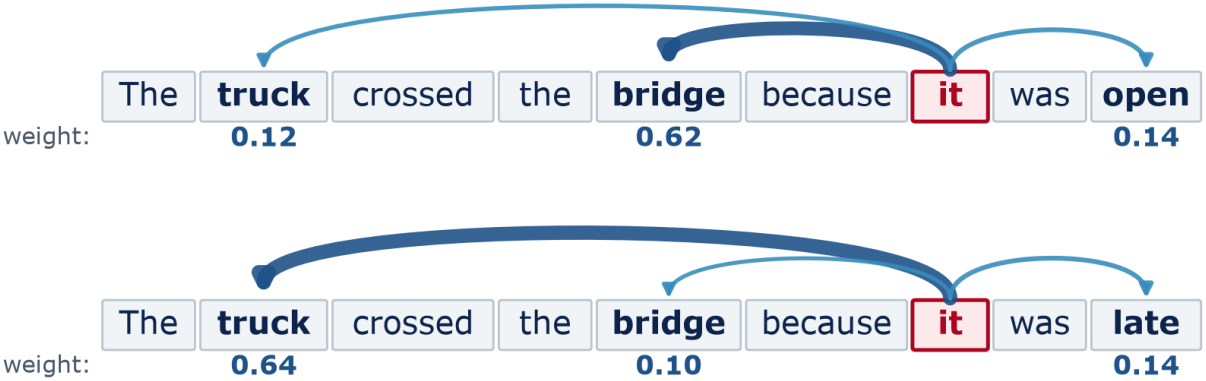
- Streamflow from rainfall and past flows
- Inputs: precipitation, temperature, past flow
- Uses: flood warning, reservoir operation

# 05

# Attention and the big picture

The bridge to generative AI

# Attention lets each word look at the words that matter




Illustrative, made-up weights for the word “it”. Two-way attention, as in BERT-style encoders; ChatGPT-style models only look back — next week: “open” looking back at “bridge”.

**Attention**

Each word scores every other word and mixes in the most relevant ones

**Transformer**

Layers of attention + small MLPs; the engine of ChatGPT-style models (Week 7)

# Choose the architecture that matches your data

| Architecture | Data it suits | Built-in idea | CE/ARCE example |
|---|---|---|---|
| **MLP** | Tables of numbers | Every input can combine with every other | Heating load from 8 geometry inputs; concrete strength from a mix |
| **CNN** | Images, grids, maps | Local patterns, same filter everywhere | Crack detection; hard hats on site; land use from satellites |
| **RNN / LSTM** | Time series, sequences | Read in order, carry a memory | Next-hour traffic; tomorrow's building load; river flow |
| **Transformer** | Text, long sequences (also images) | Attention: anything can relate to anything | Searching specs and codes; engineering assistants (Weeks 7–10) |

Attention and the big picture

# Pick the architecture

**4 min**

**Pairs · Vote**

- **1.** Predict a bridge's deck condition rating from age, traffic, and material
- **2.** Find spalling in drone photos of a parking garage
- **3.** Forecast tomorrow's hourly chiller load from the last two weeks of data
- **4.** Answer questions about a 300-page project specification
- **5.** Spot workers without hard hats in site video

For each task: MLP, CNN, RNN/LSTM, or transformer? Name the input and the output. Bonus: where might a simple non-neural model be enough?

Lab 06

# Lab day: crack detection with a CNN

**LAB**

**06**

Bring a charged laptop

Open the notebook from the course GitHub page

- Unzip the concrete photos; view them as grids of numbers
- Apply hand-made edge filters and look at the feature maps
- Transfer learning: ResNet-18 features + a small logistic-regression head
- Confusion matrix, accuracy, precision, recall — and the mistakes
- Short required part: forecast next-hour I-94 traffic with a sliding window

Coming up

# HW3 is posted today, and next week's lab day is the midterm

**HW3 — Neural Networks on Engineering Data**

- Posted today; due **end of Week 8** (Sunday 11:59 pm)
- Part A (**AI-Free**): forward pass and parameter count by hand
- MLP vs. linear regression on concrete strength; overfitting study
- Confusion matrix, feature maps, and when NOT to use a network

**Midterm exam — Week 7 lab day**

- 75 minutes, **Weeks 1–6**
- Closed book, formula sheet provided, calculator allowed
- Practice exam and formula sheet: course GitHub, `exams/` folder
- Next week's lecture (how LLMs work) is still on as usual

Wrap-up

# Key takeaways

1. Architecture = built-in assumptions about the data; match the network to the data type
2. **CNNs:** small filters slide over the image (convolution), pooling shrinks; layers learn edges → parts
3. **Transfer learning:** reuse a pretrained network and train only a small head — ideal for small datasets
4. **Sequences:** sliding windows make examples; RNNs/LSTMs carry a memory from step to step
5. **Attention** lets every element look at every other — the core of transformers and LLMs (Week 7)

Learn more

# Readings and resources

- **3Blue1Brown — But what is a convolution?** — video, 23 min: convolution with pictures
- **CNN Explainer (Georgia Tech)** — interactive: watch filters, ReLU, and pooling inside a small CNN
- **Google ML Crash Course** — neural network and embedding modules (about 1 hour each)
- **Hugging Face — Computer vision course** — Unit 2 on CNNs and transfer learning (optional, deeper)
- **Course practice midterm** — study for next week